

Державний вищий навчальний заклад
Ужгородський національний університет
Факультет інформаційних технологій
Кафедра програмного забезпечення систем

ЛАБОРАТОРНА РОБОТА № 3.1

Тема: Моделювання задачі «Рюкзак» методом динамічного
програмування

Виконав:
студент II курсу
спеціальності: 121 інженерія
програмного забезпечення
Романюк Артем Романович

Ужгород-2026

Мета: навчитися формалізувати задачу оптимізації ресурсів та розв'язувати її за допомогою методів динамічного програмування.

Побудувати оптимальне рішення для задачі «Рюкзак», проаналізувати таблицю переходів станів та візуалізувати вибраний набір предметів.

Завдання до роботи:

1. Розробка технічного завдання.
2. Вибір та опис методології (Scrum, kanban і т.д.) розробки ПЗ для методів вирішення задачі рюкзак (№1,2,4,5), розробка методики вирішення задачі. При виборі методології розробки ПЗ обрати методологію відмінну від вибору в Лабораторній роботі 1 та 2.
3. Для методу 3 вирішення задачі рюкзак - провести розробку методом динамічного програмування.
4. Розробка діаграми взаємодії для методів вирішення задачі рюкзак 1,2,3,4,5.

Хід роботи

1. Розробка технічного завдання

Технічне завдання

Зміст

1. Загальна інформація
2. Мета проекту
3. Основні функціональні вимоги

1. Загальна інформація

Назва проекту: Програмний модуль для порівняльного аналізу алгоритмів розв'язання задачі комбінаторної оптимізації (Knapsack Problem).

Галузь застосування: Навчальний процес дисципліни «Методи програмної інженерії для розв'язування прикладних задач».

Цільова платформа: Веб-середовище (браузер).

Інструментарій розробки: Мова програмування: JavaScript.

Розмітка та стилі: HTML5, CSS3.

2. Мета проекту

Метою розробки є створення програмного продукту, який дозволяє:

- Формалізувати задачу оптимізації ресурсів при заданих обмеженнях ваги та цінності предметів.
- Реалізувати та порівняти ефективність різних алгоритмічних підходів: від точних методів (перебір, ДП) до наближених (жадібний алгоритм).
- Візуалізувати внутрішні процеси обчислень, зокрема покрокову побудову таблиці станів динамічного програмування, для кращого розуміння роботи алгоритму.

3. Основні функціональні вимоги

- Організація інтерфейсу: Реалізація навігації за допомогою 5 вкладок (Tabs), кожна з яких відповідає окремому методу розв'язання задачі.
- Введення даних: Можливість задання кількості предметів (n) та максимальної місткості рюкзака (W). Введення векторів ваг ($w[i]$) та цінностей ($v[i]$) для кожного предмета.
- Алгоритмічне забезпечення (за вкладками):
 - Вкладка 1. Метод грубої сили (метод перебору): Дослідження всіх можливих комбінацій для знаходження абсолютного максимуму.
 - Вкладка 2. Рекурсивний метод: Розв'язання задачі шляхом розбиття на менші підзавдання через виклики функцій.
 - Вкладка 3. Динамічне програмування: Ітеративне заповнення матриці за рекурентним співвідношенням.
 - Вкладка 4. Жадібний алгоритм: Вибір предметів за пріоритетом співвідношення цінності до ваги.
 - Вкладка 5. Метод гілок: Побудова дерева пошуку з відсіканням неоптимальних гілок.

- Обробка та вивід результатів: Обчислення максимальної сумарної вартості предметів. Відновлення та відображення переліку предметів, що увійшли до оптимального набору.
- Візуалізація: Побудова таблиці $dp[i][w]$ для методу динамічного програмування. Анімація проміжних етапів заповнення клітинок таблиці та відновлення розв'язку.

2. Вибір та опис методології вирішення задачі. Розробка методики вирішення задачі

Для реалізації проекту обрано гнучку методологію **Scrum**. Весь обсяг роботи розділено на 6 коротких спринтів. Кожен спринт (окрім 1) фокусується на реалізації однієї конкретної вкладки та методу розв'язання задачі.

План спринтів:

Спринт 1: Розроблення базової архітектури додатку. Введення даних. Системи вкладок.

Спринт 2: Реалізація логіки «Методу грубої сили».

Спринт 3: Розробка «Рекурсивного методу».

Спринт 4: Впровадження «Динамічного програмування» та візуалізації таблиці станів.

Спринт 5: Реалізація «Жадібного алгоритму».

Спринт 6: Розробка «Методу гілок».

Методики вирішення задачі:

Метод №1: Груба сила (Brute Force)

Досліджуються всі можливі комбінації предметів (2^n варіантів). Для кожної комбінації обчислюється сумарна вага; якщо вона $< W$, фіксується сумарна вартість. Обирається варіант із максимальним значенням.

Метод №2: Рекурсивний метод

Задача розбивається на підзадачі: для кожного предмета ми або включаємо його в рюкзак, або ні. Рекурсія будує дерево рішень, обчислюючи максимум між цими двома варіантами для кожного кроку.

Метод №3: Динамічне програмування (ДП)

Замість повторних обчислень ми будуємо таблицю $dp[i][w]$. Значення в кожній клітинці - це максимальна цінність для i предметів при місткості w , яка базується на вже обчислених рішеннях для менших підзадач.

Метод №4: Жадібний алгоритм

Це евристичний метод, що обирає локально найкращий варіант на кожному кроці. Предмети сортуються за питомою вартістю ($\frac{v_i}{w_i}$), і в рюкзак додаються найбільш «вигідні» елементи, доки є вільне місце.

Метод №5: Метод гілок і меж

Покращений перебір, що використовує дерево пошуку. Для кожної гілки обчислюється верхня межа можливої вартості; якщо вона нижча за вже знайдений кращий результат, гілка «обрізається», що значно економить ресурси.

3. Для методу 3 вирішення задачі рюкзак - провести розробку методом динамічного програмування.

За допомогою рекурентного співвідношення будуємо dp :

$dp[i][w] = dp[i-1][w]$, якщо $w < w[i]$ (не можемо взяти i -й предмет)

$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w[i]] + v[i])$, якщо $w \geq w[i]$

Тут:

$dp[i-1][w]$ – максимальна цінність без i -го предмета

$dp[i-1][w-w[i]] + v[i]$ – максимальна цінність при включенні i -го предмета

Варіант 22

$n=9$, $W=19$, $w=[6, 8, 7, 1, 10, 9, 5, 3, 2]$, $v=[3, 7, 13, 4, 11, 13, 8, 10, 9]$

i/w	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	3	3	3	3	3	3	3	3	3	3	3	3	3	3
2	0	0	0	0	0	0	3	3	7	7	7	7	7	7	10	10	10	10	10	10
3	0	0	0	0	0	0	3	13	13	13	13	13	13	13	16	16	20	20	20	20
4	0	4	4	4	4	4	4	13	17	17	17	17	17	17	17	20	20	24	24	24
5	0	4	4	4	4	4	4	13	17	17	17	17	17	17	17	20	20	24	24	28
6	0	4	4	4	4	4	4	13	17	17	17	17	17	17	17	20	20	26	30	30
7	0	4	4	4	4	8	12	13	17	17	17	17	21	25	25	25	26	30	30	30
8	0	4	4	10	14	14	14	14	18	22	23	27	27	27	27	31	35	35	35	36
9	0	4	9	13	14	19	23	23	23	23	27	31	32	36	36	36	36	40	44	44

Отже: $dp[9][19] = 44$

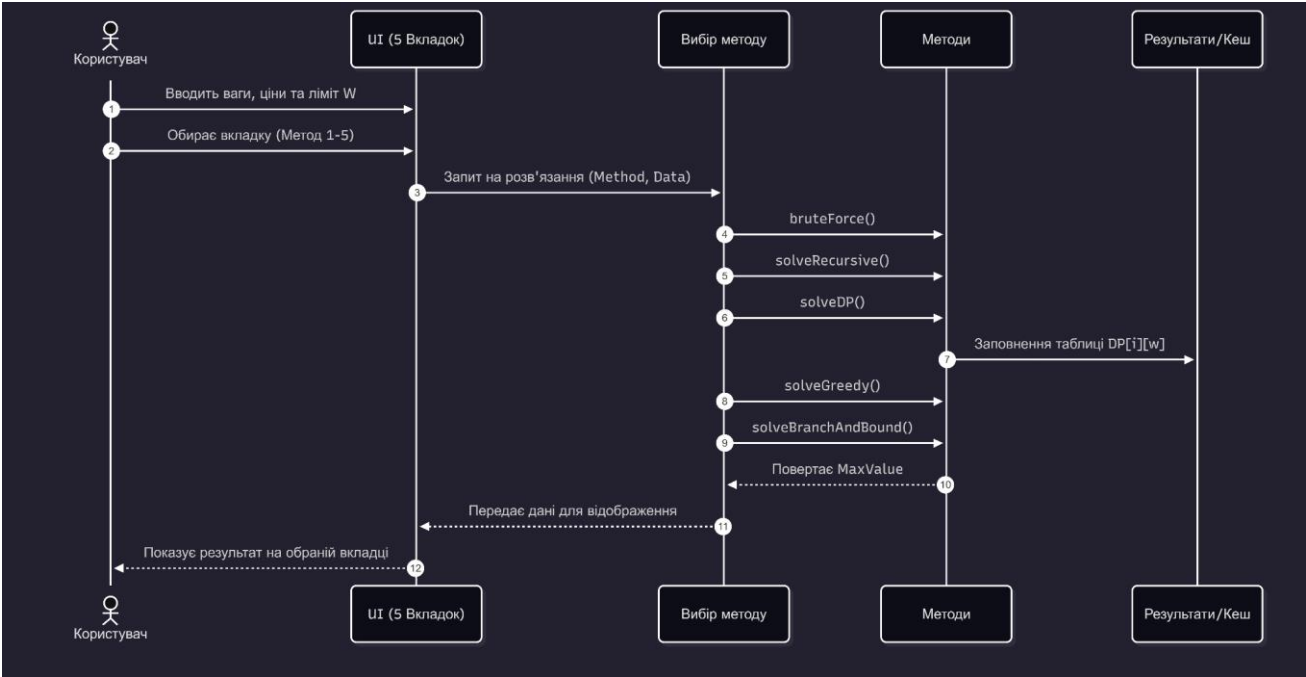
Відновлення:

якщо $dp[i][w] \neq dp[i-1][w]$, то i -й предмет входить до розв’язку;

якщо $dp[i][w] = dp[i-1][w]$, то i -й предмет не входить до розв’язку

Отже взяли: 9 - 8 - 7 - 4 - 3

4. Розробка діаграми взаємодії для методів вирішення задачі рюкзак 1,2,3,4,5



Висновки:

Під час виконання лабораторної роботи №3.1 я опанував процес формалізації задачі оптимізації ресурсів на прикладі класичної комбінаторної проблеми «Рюкзак». У ході роботи було досягнуто наступних результатів:

- **Проектування системи:** Розроблено технічне завдання для інтерактивного веб додатка, що базується на п'яти різних алгоритмічних підходах. Вибір методології **Scrum** дозволив структурувати процес розробки на шість ітераційних спринтів, що забезпечує гнучкість при реалізації кожного методу окремо.
- **Аналіз алгоритмів:** Я провів порівняльний аналіз п'яти методів вирішення задачі — від точних (повний перебір, рекурсія, динамічне програмування, метод гілок і меж) до наближених евристичних підходів (жадібний алгоритм). Це дозволило зрозуміти компроміс між обчислювальною складністю та точністю результату.
- **Практичне застосування ДП:** На прикладі Варіанту №22 я успішно реалізував метод **динамічного програмування**, побудувавши таблицю переходів станів $dp[i][w]$. Це підтвердило ефективність методу: замість експоненціального перебору 2^n варіантів, задача була розв'язана за поліноміальний час, а фінальна максимальна цінність склала **44 одиниці**.
- **Візуалізація та логіка:** Розроблена діаграма взаємодії (Sequence Diagram) чітко відображає архітектуру системи, де користувач взаємодіє з інтерфейсом, а контролер керує вибором стратегії обчислення та збереженням результатів у кеші.

Отже, я навчився не лише знаходити оптимальні рішення для складних задач, а й відновлювати перелік предметів, що входять до фінального набору, аналізуючи різницю значень у таблиці станів. Отримані знання з динамічного програмування та системного проектування є фундаментальними для подальшої розробки складного програмного забезпечення.